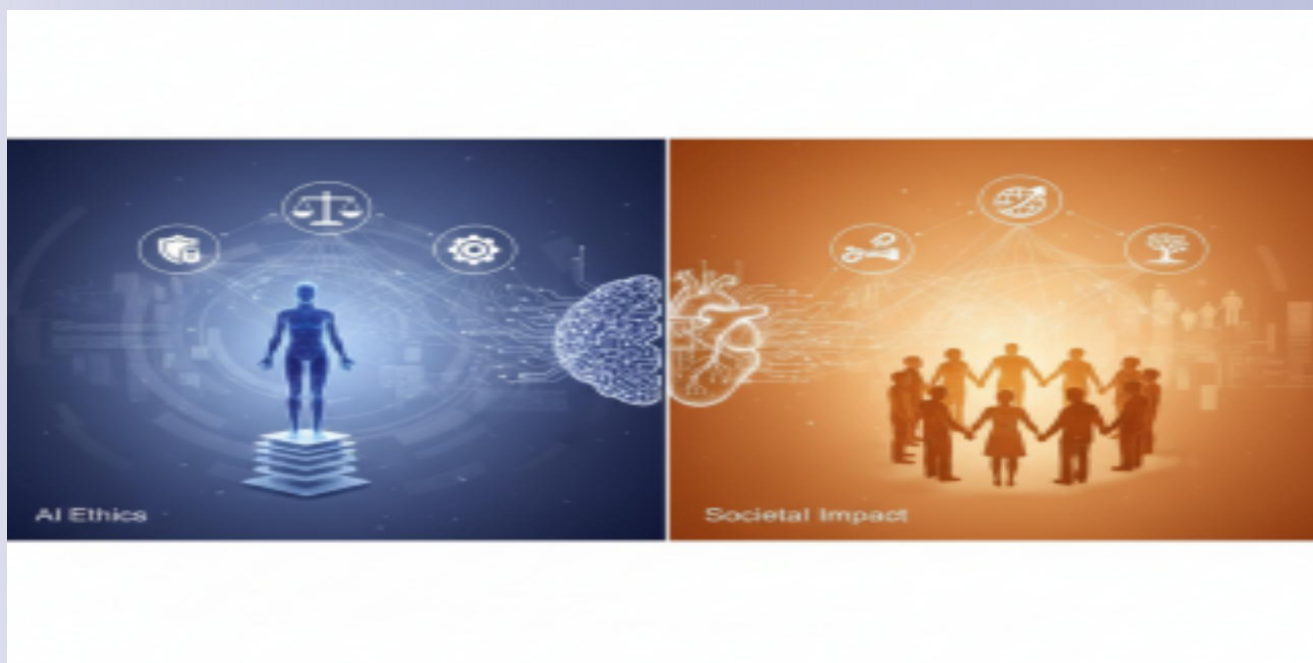
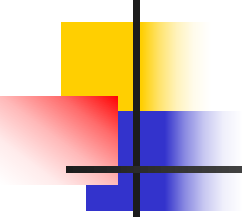


第2章 机器学习基础



湖北大学 计算机学院
王雷春



目 录

- 2.1 机器学习概述
- 2.2 数据预处理和特征工程
- 2.3 机器学习分类
- 2.4 常用监督学习算法
- 2.5 常用无监督学习算法
- 2.6 模型评估与验证
- 2.7 Scikit-learn简介
- 2.8 机器学习常见问题



2.4 常用监督学习算法

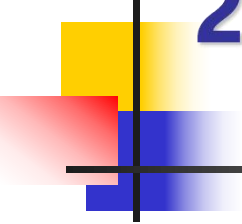
2.4.1 线性回归算法

2.4.2 K近邻算法

2.4.3 支持向量机算法

2.4.4 决策树算法





2.4.1 线性回归算法

1. 相关概念

(1) 算法含义

- ◆ 线性回归 (Linear Regression) 是统计学和机器学习中最基础、最经典的监督学习算法之一，主要用于预测连续数值型的目标变量。
- ◆ 核心思想：假设自变量（特征）与因变量（目标）之间存在线性关系，并通过拟合一条直线（或超平面）来描述这种关系。

2.4.1 线性回归算法

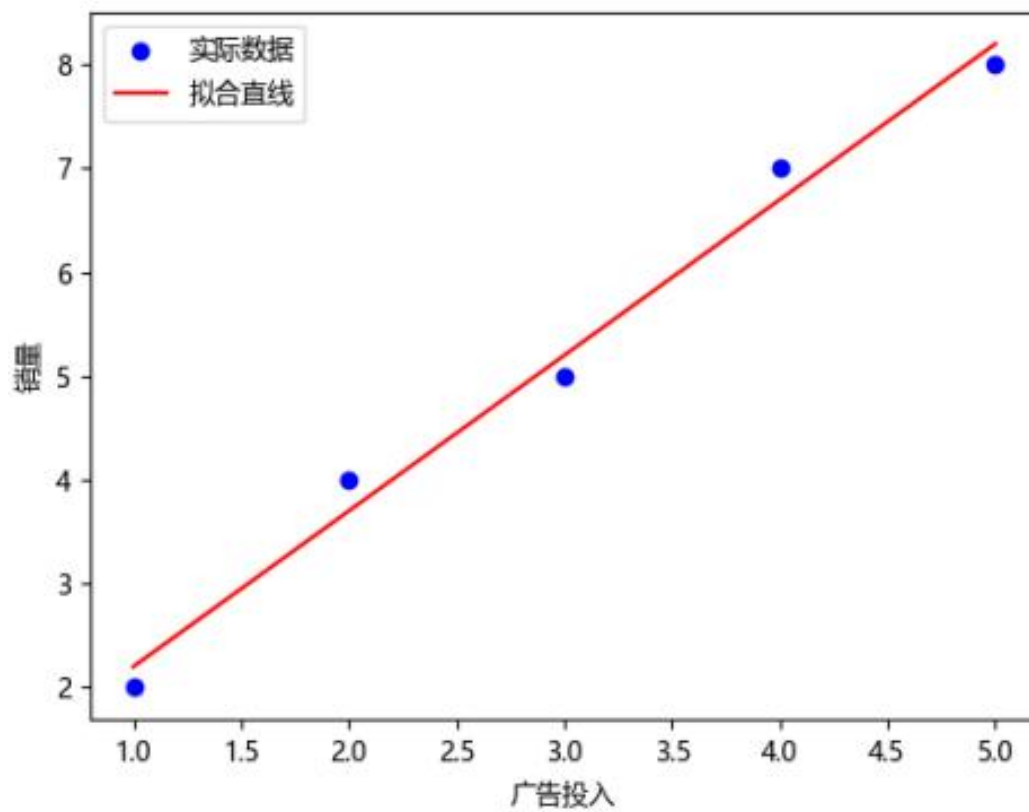
1. 相关概念

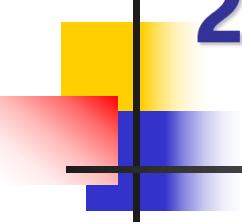
(2)形象化比喻 - 冰淇淋销量与温度

- ◆ 经营一家冰淇淋摊，发现天气越热，销量越高。
 - ◆ 数据：过去30天，记录了每天的最高温度 (X) 和冰淇淋销量 (y)。
 - ◆ 规律：温度 10°C 时卖50支， 20°C 时卖100支， 30°C 时卖150支.....几乎是线性关系。
 - ◆ 预测：明天天气预报说 25°C ，就可以用这条直线估算：明天大概能卖125支。
- ◆ 这是一种线性回归：找到温度和销量之间的线性关系，用来做业务决策（比如备货量）。

2.4.1 线性回归算法

1. 相关概念





2.4.1 线性回归算法

1. 相关概念

说明和分析:

- ◆ (1)横轴 (X轴): 表示自变量 (特征), 例如广告投入金额。
- ◆ (2)纵轴 (Y轴): 表示因变量 (目标), 例如产品销量。
- ◆ (3)散点: 每一个蓝色的点代表一个真实样本, 其位置由实际的广告投入和销量决定。
- ◆ (4)拟合直线: 红色的直线代表通过线性回归算法学习到的模型。这条直线是所有可能直线中, 能最好地描述数据整体趋势的那一条。
- ◆ (5)残差: 图中未直接画出, 但可以想象每个散点到直线的垂直距离就是该样本的预测误差 (残差)。线性回归的目标就是让所有残差的平方和最小。

2.4.1 线性回归算法

2. 基本类型

- ◆ 根据特征数量的不同，线性回归基本类型包括：一元线性回归和多元线性回归。

- ◆ (1)一元线性回归：只有一个特征，模型形式为：

$$y = \omega x + b$$

- ◆ 其中， ω 是斜率（权重）， b 是截距（偏置）。

- ◆ (2)多元线性回归：有多个特征 $x_1, x_2, \dots, x_n$ ，模型形式为：

$$y = \omega_1 x_1 + \omega_2 x_2 + \dots + \omega_n x_n + b$$

- ◆ 也可写作向量形式

$$y = W^T X + b$$

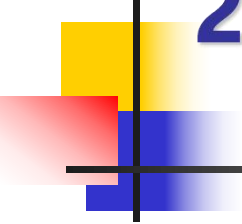
2.4.1 线性回归算法

3. 损失函数

- ◆ 线性回归通常使用均方误差作为损失函数，即预测值与真实值之差的平方的平均值：

$$L(w, b) = \frac{1}{m} \sum_{i=1}^m (y_i - (wx_i + b))^2$$

- ◆ 目标是通过调整参数 w , b 使该损失函数最小化。



2.4.1 线性回归算法

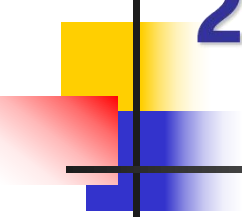
4. 求解方法

(1)最小二乘法

- ◆ 直接通过数学公式求解参数的解析解。
- ◆ 这种方法的好处是一次计算得到最优解，但是计算复杂度高（矩阵求逆），不适合特征维度极高或样本量极大的情况。

(2)梯度下降法

- ◆ 迭代优化算法，通过计算损失函数对参数的梯度，并沿着梯度的反方向更新参数，逐步逼近最小值。
- ◆ 当特征维度高或数据量大时，常用梯度下降迭代求解。



2.4.1 线性回归算法

5. 优点和缺点

(1)优点:

- ◆ 模型简单，计算效率高
- ◆ 可解释性强，能直接看出各特征的影响
- ◆ 是许多更复杂算法的基础
- ◆ 在特征与目标线性相关时，预测效果良好

(2)缺点:

- ◆ 只能拟合线性关系，对非线性数据拟合能力弱
- ◆ 对异常值非常敏感
- ◆ 特征之间若存在高度相关性，会导致模型不稳定
- ◆ 需要满足一些统计假设才能获得最佳性能

2.4.1 线性回归算法

6. 改进与变体

(1)多项式回归

- ◆ 将自变量的高次项（如 x^2 、 x^3 等）作为新特征加入模型来拟合线性关系，仍使用线性回归的求解方法，通常适用于数据呈现曲线趋势，如产量与温度的关系。数学公式如下：

$$y = \omega_1 x + \omega_2 x^1 + \dots + \omega_n x^n + b$$

(2)岭回归

- ◆ 在损失函数中加入L2正则化项（ λw_i^2 ）来防止过拟合，特别适用于特征间存在多得共线性的情况，适合高维稀疏数据。损失函数计算方法如下：

$$LOSS = MSE + \lambda \sum w_i^2$$

2.4.1 线性回归算法

6. 改进与变体

(3)Lasso回归

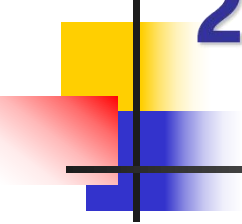
◆ 加入L1正则化项 ($\lambda|w_i|$)，不仅能防止过拟合，还能将某些特征的系数压缩为零，实现特征选择。损失函数计算方法如下：

$$LOSS = MSE + \lambda \sum |w_i|$$

(4)弹性网络

◆ 结合L1和L2正则化，当特征数量远大于样本数或特征间存在相关性时效果更好。损失函数计算方法如下：

$$LOSS = MSE + \lambda_1 \sum |w_i| + \lambda_2 \sum w_i^2$$



2.4.1 线性回归算法

7. 应用场景

线性回归算法常见应用场景如下：

- ◆ 趋势预测：如销售额随时间的变化趋势。
- ◆ 影响因素分析：研究各因素对结果的影响程度（如广告投入对销量的影响）。
- ◆ 填充缺失值：作为插值方法。
- ◆ 作为基准模型：评估复杂模型是否带来了显著提升。

2.4.2 K近邻算法

1. 相关概念

(1) 算法含义

- ◆ K近邻 (K-Nearest Neighbors, 简称KNN) 是一种非常经典且直观的监督学习算法, 属于基于实例的学习 (也称懒惰学习, lazy learning), 既可应用于分类问题, 也可用于回归问题。
- ◆ KNN的核心思想:
 - ◆ “物以类聚, 人以群分”。
 - ◆ KNN不像线性回归那样从训练数据中学习出一个明确的模型 (如权重参数), 而是直接记住所有训练样本。
 - ◆ 想知道一个新样本属于哪一类 (或它的数值是多少), 就看看在特征空间中, 离它最近的K个样本是什么情况, 然后由这些 “邻居” 来投票决定。

2.4.2 K近邻算法

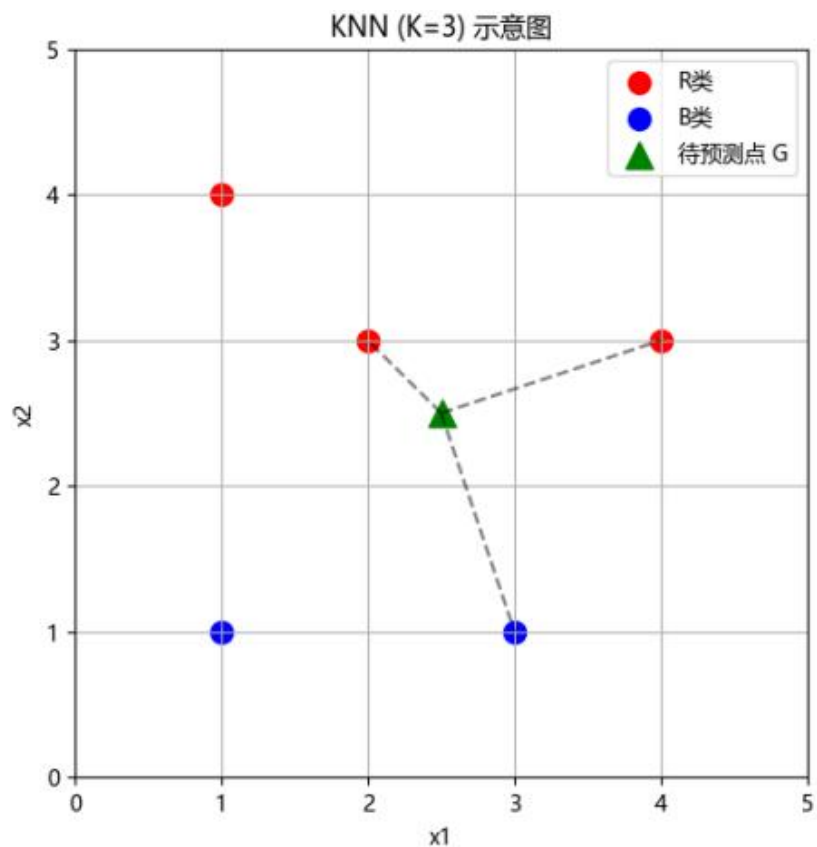
1. 相关概念

(2)形象化比喻 - 社区投票：认识一个新邻居

- ◆ 想象你刚搬进一个新小区，想知道自己家大概属于哪个收入阶层（分类任务）。于是观察周围的 K 户邻居：
 - ◆ K=3：问最近的三户邻居。如果两户是工薪阶层，一户是企业高管，那么你被划为工薪阶层（少数服从多数）。
 - ◆ K=5：扩大范围问五户。如果三户高管、两户工薪，那么你被划为企业高管。
- ◆ 对应关系：
 - ◆ 邻居 = 数据集中已有的样本（带标签）
 - ◆ 距离 = 你家到邻居家的物理距离（欧氏距离、曼哈顿距离等）
 - ◆ 投票结果 = 预测的分类标签
- ◆ K 值 = 你询问的邻居数量

2.4.2 K近邻算法

1. 相关概念



2.4.2 K近邻算法

1. 相关概念

说明和分析:

- ◆ (1)坐标轴: 横轴 x_1 , 纵轴 x_2 , 取值范围0~5。
- ◆ (2)样本点:
 - ◆ R 表示红色类样本 (坐标略)
 - ◆ B 表示蓝色类样本 (坐标略)
 - ◆ G 表示待预测的绿色点 (假设位于 (2.5, 2.5) 附近)
- ◆ (3)最近邻: 以G为中心, 计算所有样本点到G的距离, 找出最近的3个邻居。假设它们分别是:
 - ◆ 一个R (位于 (2, 3))
 - ◆ 另一个R (位于 (3, 2))
 - ◆ 一个B (位于 (2, 1))

2.4.2 K近邻算法

1. 相关概念

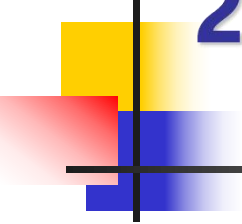
说明和分析:

- ◆ (4)投票结果: $K=3$ 时, 红色类得2票, 蓝色类得1票 $\rightarrow$ 预测G为红色类。
- ◆ 决策边界与K值的影响
 - ◆ 如果 $K=1$, G会被归为离它最近的单个邻居的类别 (可能是R或B, 取决于具体距离)。
 - ◆ 如果 $K=5$, 则可能纳入更多远处的样本, 结果可能不同。
 - ◆ K 值较小时, 决策边界更复杂 (易过拟合); K 值较大时, 边界更平滑 (可能欠拟合)。

2.4.2 K近邻算法

2. 算法主要步骤

- ◆ (1)准备数据：确定特征空间，通常需要对特征进行标准化/归一化，因为KNN依赖于距离计算，不同量纲的特征会主导距离。
- ◆ (2)选择K值：确定一个正整数K，表示要考虑的邻居数量。
- ◆ (3)计算距离：对于新样本，计算它与所有训练样本的距离（常用欧氏距离、曼哈顿距离等）。
- ◆ (4)找出K个最近邻：根据距离排序，选出距离最小的K个训练样本。
- ◆ (5)进行预测：
 - ◆ 分类：统计K个邻居的类别频次，返回频次最高的类别。
 - ◆ 回归：计算K个邻居的目标值的平均值（或加权平均），返回该值。



2.4.2 K近邻算法

3. 关键要素

(1)K值的选择

- ◆ K值较小：模型复杂度较高，容易受到噪声点的影响，发生过拟合（决策边界变得曲折）。
- ◆ K值较大：模型复杂度降低，决策边界更平滑，但可能包含距离较远的其他类别的点，导致欠拟合。
- ◆ 通常做法：通过交叉验证选择最优的K值，一般取奇数（分类时避免平局）。

2.4.2 K近邻算法

3. 关键要素

(2)距离度量

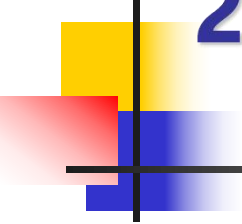
- ◆ 欧氏距离：最常用，适用于连续数值型特征。

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

- ◆ 曼哈顿距离：适用于高维空间或某些特殊场景。

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n |x_i - y_i|$$

- ◆ 闵可夫斯基距离：欧氏距离和曼哈顿距离的一般化形式。

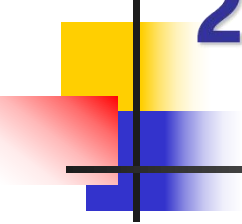


2.4.2 K近邻算法

3. 关键要素

(3)特征归一化

- ◆ 原因：由于KNN基于距离，如果某个特征取值范围很大（如收入0-100万），而另一个特征取值范围很小（如年龄0-100），那么前者将主导距离计算，导致后者失效。
- ◆ 通常需要对特征进行归一化（缩放到 $[0,1]$ ）或标准化（均值为0，标准差为1）。

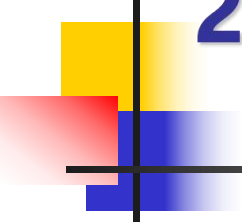


2.4.2 K近邻算法

4. 优点和缺点

优点:

- ◆ 简单直观：算法易于理解，容易实现。
- ◆ 无需训练：属于懒惰学习，训练阶段只是存储数据，几乎没有时间成本。
- ◆ 适用于多分类：天然支持多分类问题。
- ◆ 没有对数据的分布假设：可以处理复杂的决策边界。

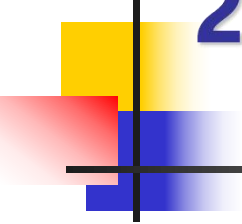


2.4.2 K近邻算法

4. 优点和缺点

缺点：

- ◆ 计算开销大：预测时需要计算与所有训练样本的距离，当训练集很大时，预测速度很慢。
- ◆ 高维灾难：随着特征维度增加，数据在空间中变得稀疏，距离度量意义减弱，效果下降。
- ◆ 对数据尺度敏感：必须进行归一化处理。
- ◆ 对噪声敏感：尤其是K值较小时，异常点会严重影响预测结果。

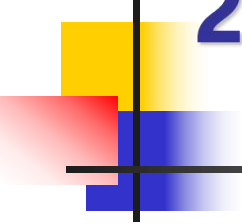


2.4.2 K近邻算法

5. 改进与变体

KNN常见改进与变体如下：

- ◆ 加权KNN：根据距离远近赋予邻居不同的投票权重，距离越近的邻居权重越大，改善预测效果。
- ◆ 半径搜索：指定一个半径，只考虑该半径内的邻居，适用于样本分布不均的情况。
- ◆ KD树 / 球树：通过构建树形数据结构加速最近邻搜索，降低预测时的计算复杂度。



2.4.2 K近邻算法

6. 应用场景

K近邻算法常见应用场景如下：

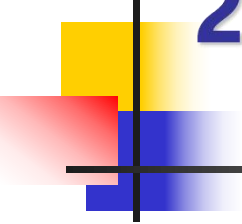
- ◆ 小型数据集：样本量不大（几千到几万），特征维度适中。
- ◆ 多分类问题：类别数量较多，且决策边界复杂。
- ◆ 推荐系统：基于用户或物品的相似性进行推荐（协同过滤的一种实现）。
- ◆ 作为基准模型：快速搭建一个简单模型，验证数据质量和特征工程效果。

2.4.3 支持向量机算法

1. 相关概念

(1) 算法含义

- ◆ 支持向量机 (Support Vector Machine, SVM) 是一种经典的监督学习算法，主要用于分类问题，也可用于回归（此时称为支持向量回归，SVR）。
- ◆ SVM核心思想：
 - ◆ 在特征空间中寻找一个最优的决策边界（超平面），使得不同类别的样本不仅能够被正确分开，而且离边界最近的样本（即支持向量）到该边界的间隔最大化。
 - ◆ 这种最大化间隔的策略赋予了SVM良好的泛化能力。

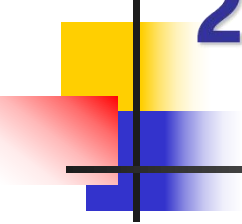


2.4.3 支持向量机算法

1. 相关概念

(1) 算法含义

- ◆ SVM通过最大化分类间隔和引入核技巧，实现了强大的分类能力，尤其擅长处理高维小样本数据。
- ◆ SVM调参复杂且在大数据上训练较慢，但它依然是机器学习领域的重要基石，在众多实际应用中表现出色。



2.4.3 支持向量机算法

1. 相关概念

(2)形象化比喻 - 国境线与界碑

- ◆ 把两类数据想象成两个相邻的国家。
 - ◆ 分界线：就是国境线。
 - ◆ 最大间隔：两个国家都想在边境留出一定的缓冲区（比如非军事区）。SVM 的目标就是让这个缓冲区尽可能宽，避免以后误入对方领土（减少分类误差）。
 - ◆ 支持向量：就是双方在边境线上设立的界碑。界碑的位置直接决定了国境线的走向。如果移走一块界碑，国境线可能就会移动。

2.4.3 支持向量机算法

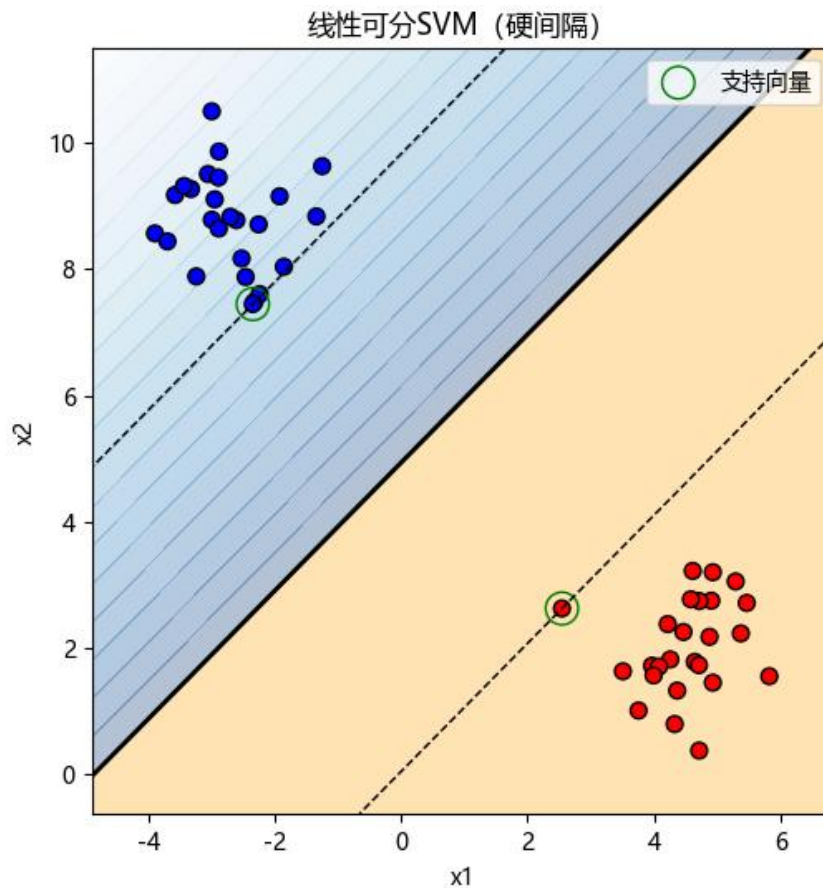
2. 算法类型

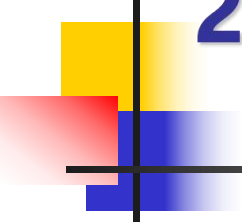
(1) 线性可分SVM（硬间隔）

- ◆ 线性可分SVM（硬间隔）是支持向量机最基础的形式，它要求训练数据在特征空间中完全线性可分，并且强制所有样本点不仅被正确分类，还必须位于间隔边界之外。
- ◆ 目标是找到使几何间隔最大的唯一最优超平面 $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$
- ◆ 特点：
 - ◆ 适用于训练数据在原始空间中完全线性可分的情况。
 - ◆ 要求所有样本必须被正确分类，且位于间隔边界之外。
 - ◆ 对异常点非常敏感。

2.4.3 支持向量机算法

2. 算法类型



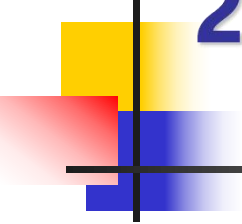


2.4.3 支持向量机算法

2. 算法类型

说明和分析：

- ◆ 实线：最优分类超平面，将两类完美分开。
- ◆ 虚线：间隔边界，位于决策平面两侧距离为1的位置。
- ◆ 绿色圆圈：支持向量，它们恰好落在虚线上（因为数据线性可分且C极大，没有样本违反间隔）。
- ◆ 所有样本点都位于虚线之外（或恰在虚线上），满足硬间隔约束。



2.4.3 支持向量机算法

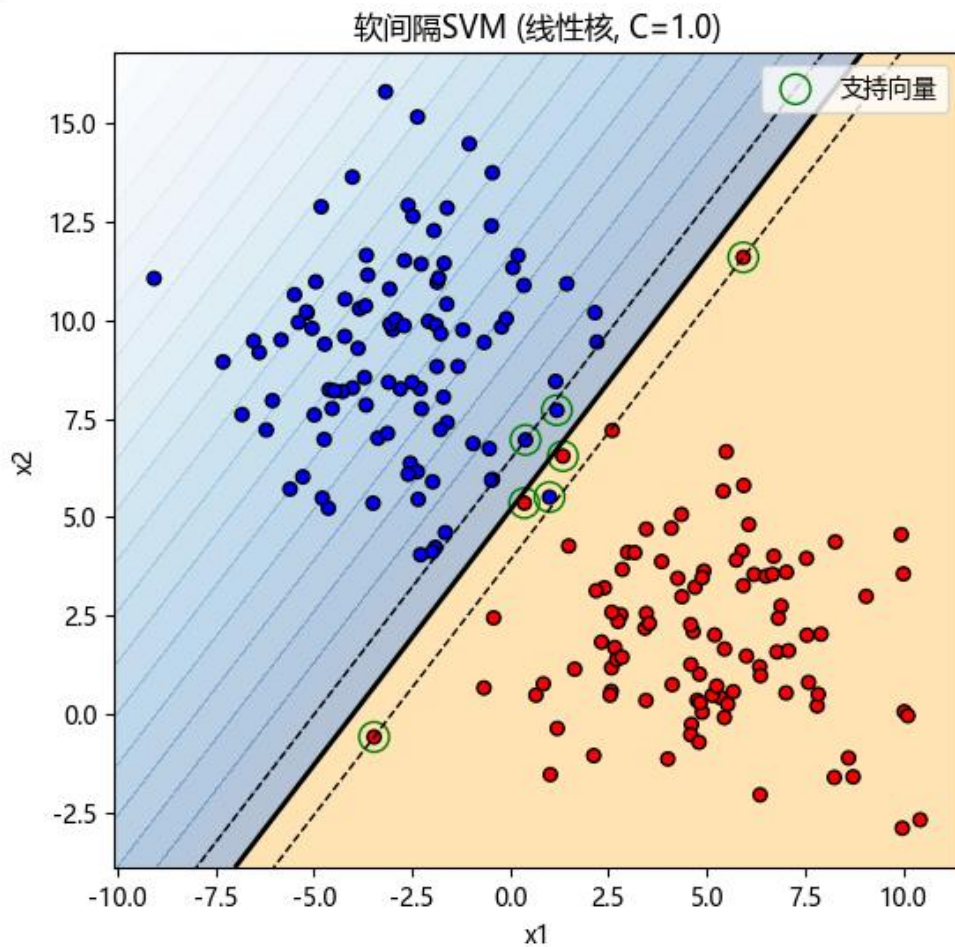
2. 算法类型

(2) 线性SVM（软间隔）

- ◆ 软间隔SVM是支持向量机在实际应用中最常用的形式。
- ◆ 通过在硬间隔SVM的基础上引入松弛变量和惩罚参数，允许部分样本点违反间隔约束（即越过间隔边界甚至被错分），从而增强模型对噪声和线性不可分数据的适应能力，并防止过拟合。
- ◆ 引入松弛变量和惩罚参数 C ，允许部分样本被错分或落在间隔边界内。
- ◆ C 越大，对错误分类的惩罚越重，决策边界越复杂（可能过拟合）； C 越小，对错误的容忍度越高，边界越平滑（可能欠拟合）。
- ◆ 这是实际中最常用的线性SVM形式。

2.4.3 支持向量机算法

2. 算法类型

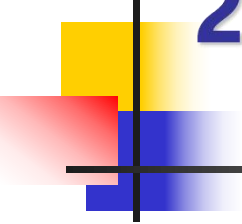


2.4.3 支持向量机算法

2. 算法类型

说明和分析：

- 图中可以看到实线决策边界和两条虚线间隔边界。
 - 许多样本点落在了两条虚线之间（即间隔内部）甚至跨越了实线（被错分）。这些点都对应着 $\xi_i > 0$ 。
 - 所有被绿色圆圈包围的点都是支持向量，包括：位于虚线边界上的点（ $\xi_i = 0$ ）、虚线内部的点（ $0 < \xi_i < 1$ ）、以及错分点（ $\xi_i \geq 1$ ）。它们共同决定了模型。
- ◆ 修改代码中的 C 值，可以直观看到软间隔SVM的决策边界如何变化：
- ◆ C 很小（如 0.01）：对违规惩罚小，间隔很宽，许多点被允许在间隔内甚至错分，决策边界平滑，但可能欠拟合（训练误差大）。
 - ◆ C 很大（如 100）：对违规惩罚大，间隔变窄，尽可能让所有点正确分类，决策边界复杂，容易过拟合（对噪声敏感）。



2.4.3 支持向量机算法

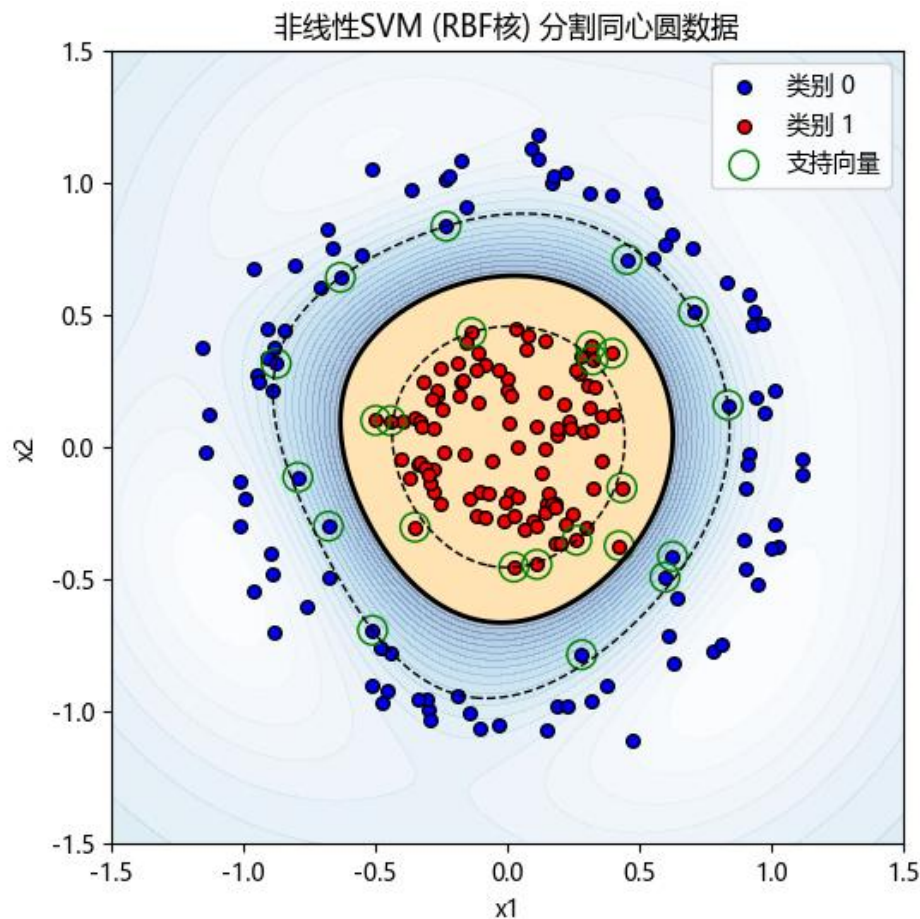
2. 算法类型

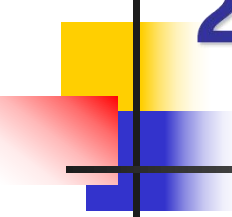
(3)非线性SVM (核技巧)

- ◆ 当数据在原始空间线性不可分时，使用核函数将数据映射到高维特征空间，然后在高维空间寻找线性超平面。
- ◆ 非线性SVM的核心在于核技巧：当数据在原始特征空间中线性不可分时，通过核函数将数据隐式映射到一个更高维的空间，使得数据在该高维空间中变得线性可分，然后在这个高维空间中寻找最优超平面。映射回原始空间后，决策边界就呈现出非线性形态。
- ◆ 常用核函数包括：线性核、多项式核、径向基函数核、Sigmoid核。

2.4.3 支持向量机算法

2. 算法类型



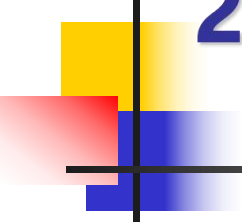


2.4.3 支持向量机算法

2. 算法类型

说明和分析:

- ◆ (1)决策边界：中间的实线是决策边界 ($\text{level}=0$)，它将平面划分为蓝色区域和橙色区域。
- ◆ (2)间隔边界：两条虚线 ($\text{level}=-1$ 和 $\text{level}=1$) 是间隔边界。位于这两条虚线上或之间的样本（除了被错分的点）是支持向量。
- ◆ (3)支持向量：用绿色空心圆圈标出的点。它们位于间隔边界上或跨越了边界，是唯一影响模型的样本。
- ◆ (4)背景色：蓝色区域表示决策函数值 < 0 （预测为类别0），橙色区域表示决策函数值 > 0 （预测为类别1）。



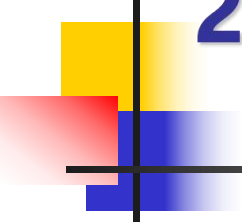
2.4.3 支持向量机算法

2. 算法类型

说明和分析:

◆ (5)关键观察:

- ◆ 非线性分割: 决策边界是一个闭合的环状曲线, 成功将内圈的红色类与外圈的蓝色类分开。
- ◆ 间隔最大化: 两条虚线之间的区域 (间隔) 尽可能宽, 且中间实线位于正中间。
- ◆ 支持向量稀疏性: 只有少数样本 (靠近边界或错分的) 被选为支持向量, 其他远离边界的样本不影响模型。



2.4.3 支持向量机算法

3. 关键要素

(1) 惩罚参数 C

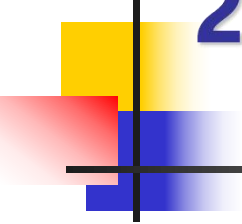
- ◆ 控制对错误分类的容忍度。 C 越大，模型越倾向于将训练样本完全分类正确（可能过拟合）； C 越小，决策边界越平滑，泛化能力可能更强。

(2) 核函数

- ◆ 包括：线性核、多项式核、径向基函数核等。

(3) 核函数参数（如 RBF 核的 γ ）

- ◆ γ 定义了单个样本的影响半径。
- ◆ γ 越大，支持向量影响的范围越小，决策边界越曲折（易过拟合）； γ 越小，影响范围越大，边界越平滑。



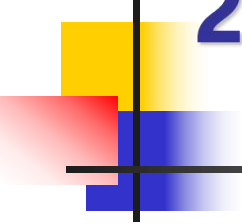
2.4.3 支持向量机算法

4. 求解方法

SVM的求解通常转化为一个凸二次规划问题，有全局最优解。

常用方法包括：

- ◆ (1)序列最小优化：将大优化问题分解为多个小问题，每次优化两个变量，高效求解。
- ◆ (2)梯度下降/随机梯度下降：用于大规模数据的近似求解。

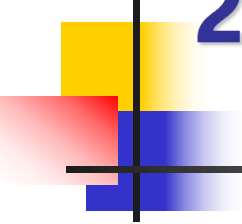


2.4.3 支持向量机算法

5. 优点和缺点

优点:

- ◆ 在高维空间表现良好：尤其适合特征数大于样本数的情况。
- ◆ 基于支持向量：只依赖少量支持向量，节省内存。
- ◆ 核技巧强大：可以处理非线性分类。
- ◆ 泛化能力强：最大化间隔策略降低了过拟合风险。

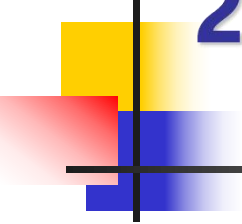


2.4.3 支持向量机算法

5. 优点和缺点

缺点：

- ◆ 对大规模数据训练慢：传统求解复杂度高，SMO等算法在大数据上仍较慢。
- ◆ 对参数和核函数敏感：C、 γ 等需要仔细调参，否则性能受影响。
- ◆ 黑盒性：模型可解释性较弱（尤其在非线性核下）。
- ◆ 对缺失数据敏感：需要预先处理缺失值。

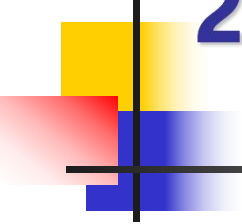


2.4.3 支持向量机算法

6. 应用场景

支持向量机算法常见应用场景如下：

- ◆ 文本分类：如垃圾邮件识别，特征空间稀疏且维度高。
- ◆ 图像识别：小规模图像分类，结合特征提取效果不错。
- ◆ 生物信息学：如基因表达数据分类，样本少特征多。
- ◆ 手写数字识别：经典应用案例。



2.4.4 决策树算法

1. 相关概念

(1) 算法含义

- ◆ 决策树算法是一种非常流行且直观的机器学习算法，属于监督学习的范畴。
- ◆ 既可以用于分类任务（如预测类别，如是否垃圾邮件），也可以用于回归任务（如预测房价）。
- ◆ 简单来说，它的工作原理就是模仿人类做决定时的思考流程，通过一系列“如果...那么...”的规则，将数据不断划分，最终得出结论。

2.4.4 决策树算法

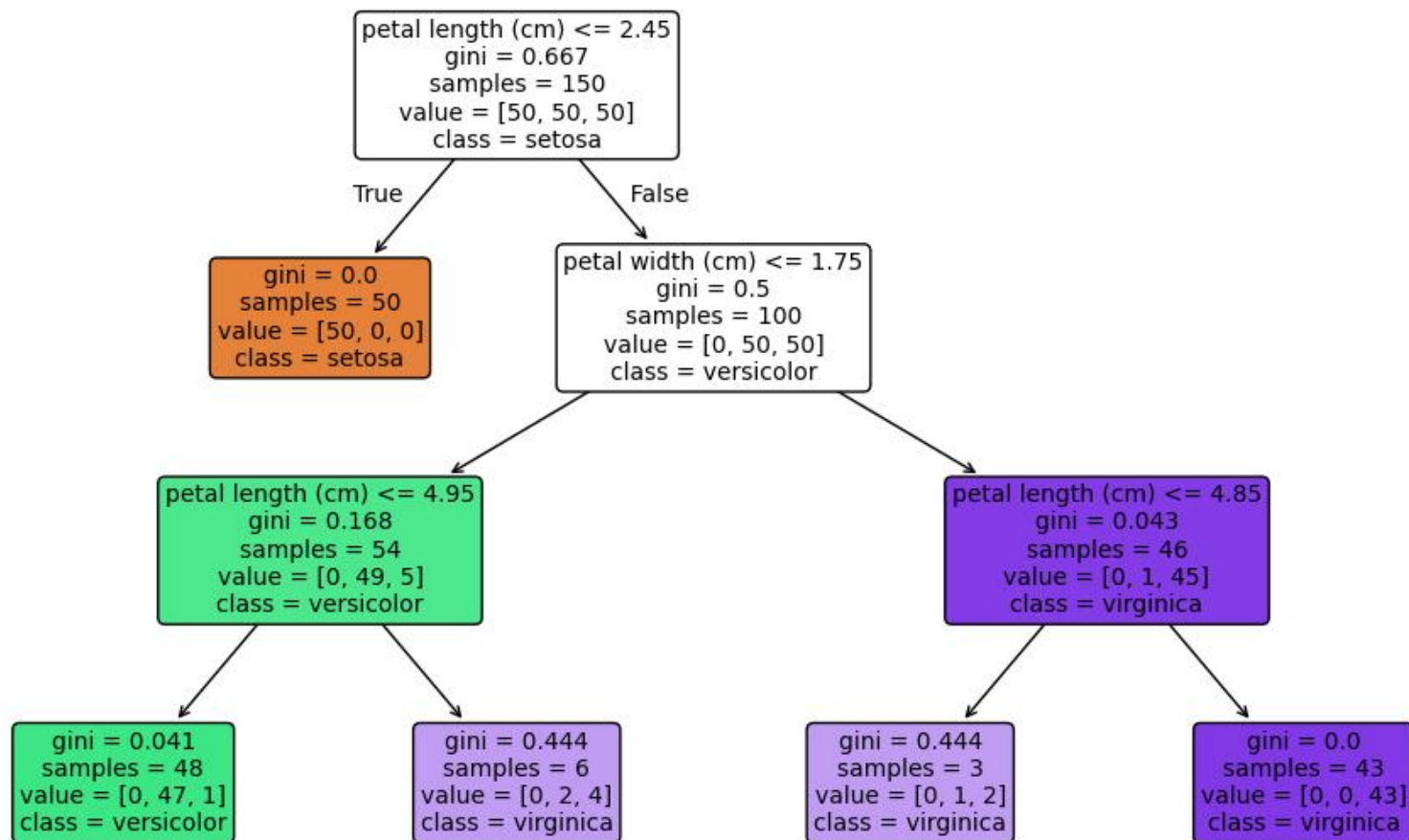
1. 相关概念

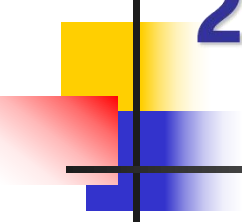
(2)形象化比喻 - 医生诊断流程

- ◆ 病人走进诊室，医生通过一系列症状判断病情：
- ◆ ①是否发烧？ → 是 → 测量体温
- ◆ ②体温 $> 38.5^{\circ}\text{C}$ ？ → 是 → 检查喉咙
- ◆ ③喉咙红肿？ → 是 → 诊断：扁桃体炎
- ◆ ④否则 → 可能是普通感冒
- ◆ 决策树就是一张结构化的“诊断流程图”。关键点：
 - ◆ 每一步只问一个最关键的症状（特征）
 - ◆ 不同的回答走向不同的检查路径
 - ◆ 最终落在某个诊断（类别）上

2.4.4 决策树算法

1. 相关概念





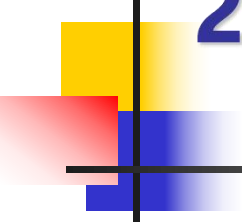
2.4.4 决策树算法

1. 相关概念

说明和分析:

在窗口中显示一棵彩色的树，每个节点包含：

- ◆ 划分条件 (如 $\text{petal width (cm)} \leq 0.8$)
- ◆ 基尼系数 (或熵)
- ◆ 样本数
- ◆ 各类别样本数
- ◆ 类别结论



2.4.4 决策树算法

2. 决策树构成

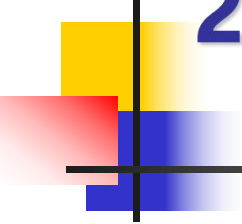
一棵决策树构成：

- ◆ 根节点：最顶端的第一个判断点，代表整个数据集。
- ◆ 内部节点：中间判断点，代表对一个特征的测试（如“温度是否大于30度？”）。
- ◆ 叶节点：最终的结论，代表一个类别（分类）或一个数值（回归）。
- ◆ 分支：判断的结果（如“是”或“否”），连接着不同的节点。

2.4.4 决策树算法

3. 算法核心

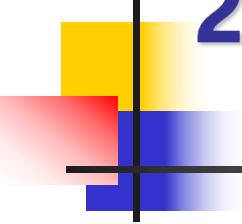
- ◆ 建树关键是决定先判断哪个特征。
- ◆ 算法目标是让每一次划分后，子节点数据“纯度”越来越高。
- ◆ 衡量“纯度”有三种指标：
 - ◆ (1)信息增益：基于信息论的概念。划分前后“不确定性”减少得越多，信息增益越大。算法优先选择信息增益最大特征进行划分。
 - ◆ (2)信息增益率：对信息增益的改进，通过除以特征自身的固有值来惩罚取值过多的特征，从而避免上述缺点。是 C4.5 算法的核心。
 - ◆ (3)基尼系数：衡量数据集的不纯度。基尼系数越低，纯度越高。算法会优先选择划分后基尼系数最小的特征。是 CART 算法（分类与回归树）的核心。



2.4.4 决策树算法

4. 算法类型

- ◆ (1)ID3算法。核心：使用信息增益作为特征选择标准。特点：算法简单，但只能处理离散型特征，且容易过拟合。
- ◆ (2)C4.5算法。核心：使用信息增益率作为特征选择标准。特点：ID3的改进版，可以处理连续型特征和缺失值，生成的规则也更易于理解，在数据挖掘领域具有里程碑式的地位。
- ◆ (3)CART算法。核心：分类树使用基尼系数最小化准则，回归树使用均方误差 (MSE) 最小化准则。特点：生成的树是二叉树，结构简洁。既可以做分类也可以做回归，是目前最主流的决策树算法之一。

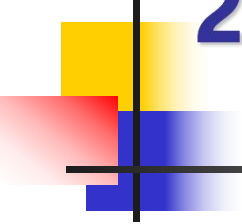


2.4.4 决策树算法

5. 优点和缺点

优点:

- ◆ 直观易懂：可视化后非常直观，解释性强，甚至能帮助发现新的数据模式。
- ◆ 预处理少：对数据缩放（归一化）不敏感，也不需要像神经网络那样做复杂的特征工程。
- ◆ 能处理混合数据：既可以处理数值型数据（如年龄），也可以处理类别型数据（如颜色）。

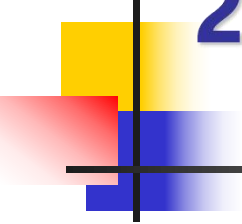


2.4.4 决策树算法

5. 优点和缺点

缺点：

- ◆ 容易过拟合：如果树长得太深、太茂盛，可能会把训练数据中的噪声也学进去，导致在新数据上表现不佳。（常用解决方法：剪枝、设置最大深度）。
- ◆ 不稳定：数据的微小变化可能导致生成完全不同的树。
- ◆ 容易产生偏差：如果某些类别的样本量远大于其他类别，树可能会偏向这些多数类。



2.4.4 决策树算法

6. 应用场景

支持向量机算法常见应用场景如下：

- ◆ 信用评估：根据收入、负债、信用历史判断是否放贷。
- ◆ 医疗诊断：根据症状、检查结果辅助判断疾病。
- ◆ 用户分类：根据浏览行为、消费记录预测用户是否会购买。
- ◆ 工业检测：根据传感器数据判断机器是否故障。

2.5 常用无监督学习算法

2.5.1 K-Means算法

2.5.2 层次聚类算法

2.5.3 DBSCAN算法

2.5.4 PCA算法

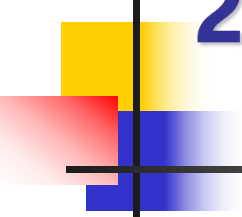


2.5.1 K-Means算法

1. 相关概念

(1) 算法含义

- ◆ K-means 是一种最经典、最常用的无监督学习算法，主要用于聚类问题。目标是把数据集中的样本划分成 K 个不同的簇（cluster），使得同一簇内样本尽可能相似，不同簇样本尽可能不同。
- ◆ K-means 的核心思想：找到 K 个中心点，让所有样本到其所属簇中心的距离平方和最小，从而实现数据的自动分组。
 - ◆ (1) 簇中心（centroid）：每个簇用一个虚拟点表示，通常取该簇所有样本的均值。
 - ◆ (2) 相似度度量：通常使用欧氏距离，样本离哪个中心近就属于哪个簇。
 - ◆ 目标：最小化簇内误差平方和（SSE），也称惯性（inertia）。



2.5.1 K-Means算法

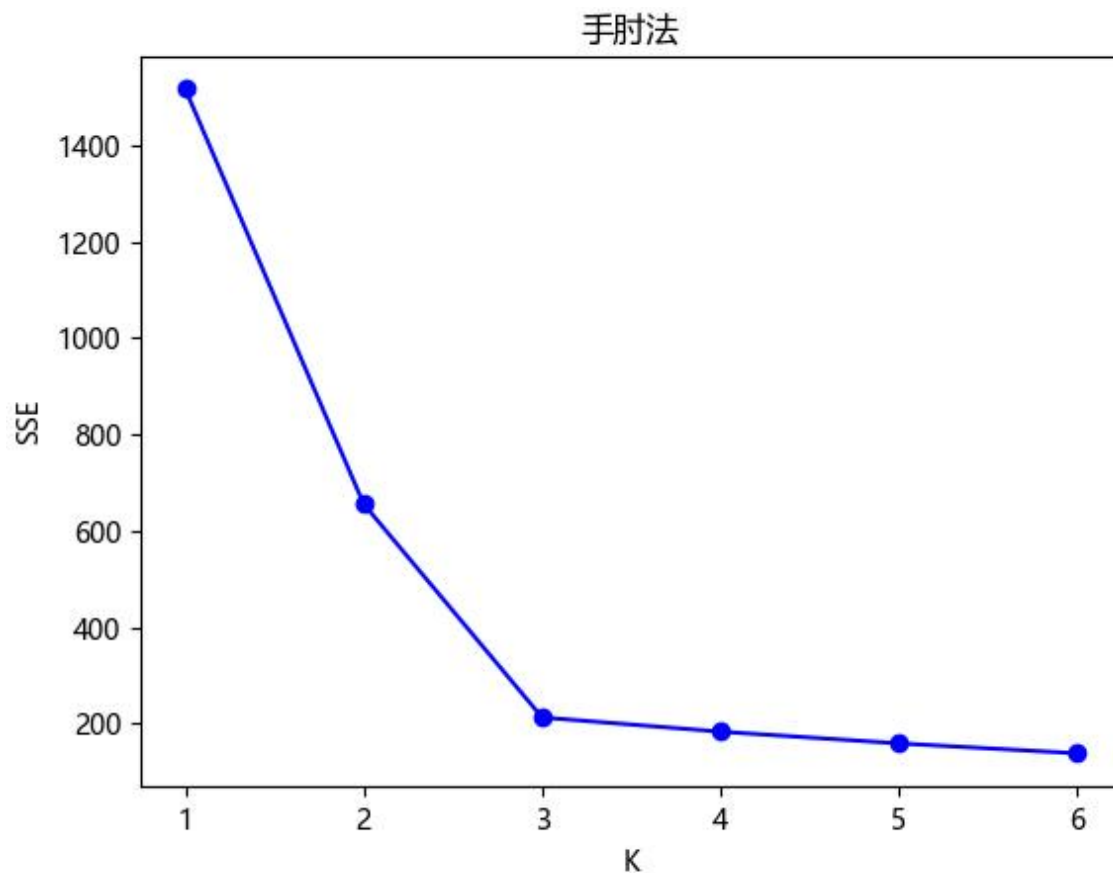
1. 相关概念

(2)形象化比喻：老师分组教学

- ◆ 课堂上，老师想把学生按成绩水平分成 K 个小组（比如：基础组、提高组、拔尖组）。
- ◆ 初始：老师随便点了几个学生当组长（初始中心）。
- ◆ 分组：其他学生坐到分数和自己最接近的组长旁边（按距离分配）。
- ◆ 测验：老师让每个小组算出自己的平均分（更新中心）。
- ◆ 重组：老师根据新的平均分，重新调整座位，让分数靠近新平均分的学生坐一起。
- ◆ 结果：最终每个小组的平均分差异明显，组内成员分数相近。

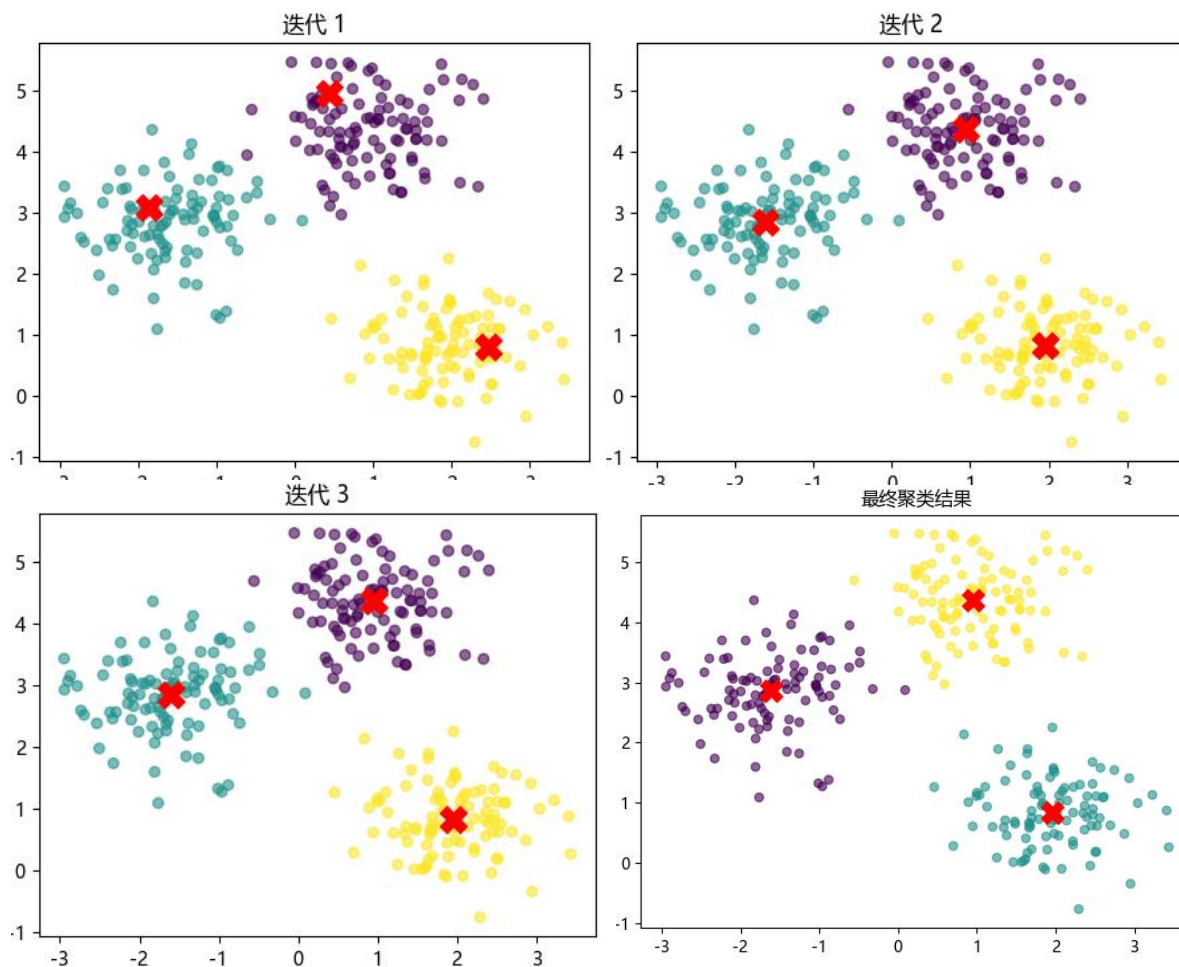
2.5.1 K-Means算法

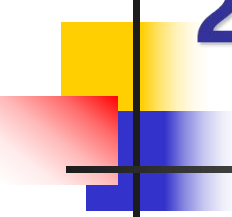
1. 相关概念



2.5.1 K-Means算法

1. 相关概念



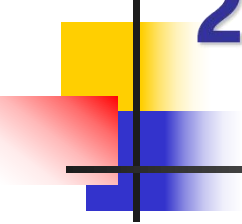


2.5.1 K-Means算法

1. 相关概念

说明和分析:

- ◆ (1)生成 300 个样本点，分布在 3 个簇中（真实标签已知，仅用于验证）。
- ◆ (2)通过手肘图确认 $K=3$ 是合适的。
- ◆ (3)使用 KMeans 的 `init` 参数传入自定义初始中心，并逐步执行“分配-更新”循环。
- ◆ (4)画出每次迭代后的 SSE（误差平方和），看它如何快速下降并趋于平稳。
- ◆ (5)使用 sklearn 的 KMeans 快速得到最终结果



2.5.1 K-Means算法

2. 算法主要步骤

K-Means 算法的主要步骤：

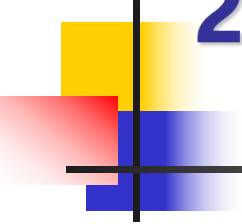
- ◆ (1)确定簇的数量 K 。用户需事先指定要将数据分为几类。例如，若想将客户分为高、中、低三类，则 $K=3$ 。这个值可以通过先验知识或后续的评估方法（如肘部法则）确定。
- ◆ (2)初始化簇中心。随机从数据集中选取 K 个点作为初始的聚类中心（质心）。由于是随机选择，不同运行可能产生不同结果，因此实践中常多次运行取最优。
- ◆ (3)分配数据点。计算每个数据点到所有 K 个质心的距离，将其分配给距离最近的那个质心所对应的簇。

2.5.1 K-Means算法

2. 算法主要步骤

K-Means 算法的主要步骤:

- ◆ (4)更新质心。对于每一个簇，重新计算其所有成员点的平均位置，并将该位置设为新的质心。
- ◆ (5)迭代直至收敛。重复执行(3)和(4)两个步骤，直到满足以下任一停止条件：
 - ◆ 所有质心的位置不再发生明显变化；
 - ◆ 数据点的簇归属不再变动；
 - ◆ 达到预设的最大迭代次数。
- ◆ 最终输出为 K 个簇及其对应的质心位置，完成数据的自动分组。

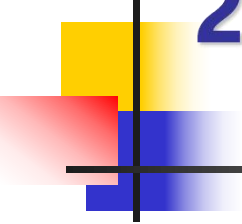


2.5.1 K-Means算法

3. K值确定方法

常用方法：

- ◆ (1)手肘法 (Elbow Method) : 绘制 K 与 SSE 的关系图, 选择 SSE 下降变缓的拐点处的 K。
- ◆ (2)轮廓系数 (Silhouette Coefficient) : 衡量簇内紧密度和簇间分离度, 选择平均轮廓系数最大的 K。
- ◆ (3)Gap 统计量: 比较原始数据 SSE 与均匀分布下的 SSE, 选择 Gap 最大的 K。
- ◆ (4)业务经验: 根据实际需求确定 K (如分 5 类客户便于运营)。

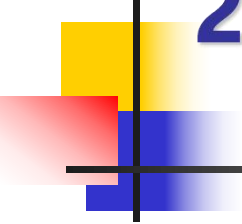


2.5.1 K-Means算法

4. 优点和缺点

优点:

- ◆ 算法简单高效：原理直观、易于理解和实现，时间和空间复杂度较低，适用于大规模数据集，且数据量越大越能体现其效率优势。
- ◆ 聚类效果明确：当数据簇呈密集、球状或团状分布且簇间差异明显时，能生成紧密的簇，聚类效果较好。
- ◆ 可扩展性强：对处理大数据集具有良好的伸缩性，能适应多种应用环境。

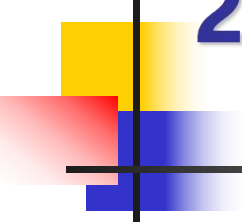


2.5.1 K-Means算法

4. 优点和缺点

缺点：

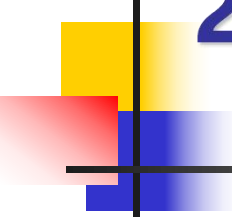
- ◆ 需预设簇数量K：K值依赖人工经验或试错法确定，经验不足时易导致簇划分过细或过粗，尤其在无先验知识的复杂场景中盲目性强。
- ◆ 对初始质心敏感：随机选择初始质心可能导致结果陷入局部最优，不同初始值可能产生不同聚类结果。
- ◆ 数据类型与分布限制：仅适用于数值型数据，不适用于分类属性数据；对非凸形状、大小差异大的簇或噪声、孤立点敏感，少量异常数据会显著影响质心计算。



2.5.1 K-Means算法

5. 改进与变体

- ◆ K-means++：改进初始化，随机选择相距较远的初始中心，大幅提升结果质量和收敛速度（sklearn 默认采用）。
- ◆ ISODATA：动态调整 K，可合并、分裂簇。
- ◆ Mini-batch K-means：每次用部分样本更新中心，适合超大规模数据。
- ◆ K-medoids：用真实样本点作为中心（中位数），对异常值更鲁棒。
- ◆ 核 K-means：通过核技巧处理非线性簇结构。

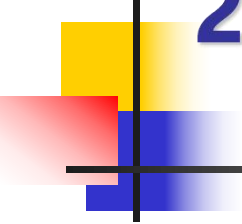


2.5.1 K-Means算法

6. 应用场景

K-Means算法常见应用场景如下：

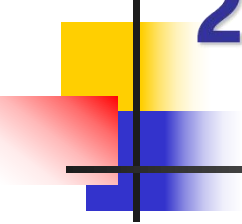
- ◆ 客户细分与市场营销。这是 K-means 最经典的应用之一。例如，企业利用用户的行为数据（如消费金额、频率、浏览时长、年龄等）进行聚类。
- ◆ 图像分割与压缩。例如，在计算机视觉领域，K-means 常被用作图像预处理工具。
- ◆ 异常检测。例如，在网络安全、金融风控等领域，K-means 可以用来识别离群点。
- ◆ 文档聚类与文本挖掘。例如，利用自然语言处理技术将文本转化为向量（如 TF-IDF 或 Word2Vec）后，可以对海量文档进行聚类。



2.5.1 K-Means算法

6. 应用场景

- ◆ 特征工程与数据降维。例如，在机器学习建模前的数据预处理阶段，K-means 常被用来增强模型的表达能力。
- ◆ 社交网络分析。例如，根据用户的兴趣爱好、关注列表或交互关系（嵌入向量）进行聚类。
- ◆ 生物学与医学。例如，在基因表达数据分析中，对具有相似表达模式的基因进行聚类，帮助发现功能相关的基因家族；在医学影像分析中，辅助识别细胞类型或病灶区域。
- ◆ 物流与地理位置分析。例如，外卖平台将商家与用户聚合，确定“配送区域”的划分；共享单车平台识别“热点停车区”；物流公司优化“仓库选址”和配送路线规划。



2.5.2 层次聚类算法

1. 相关概念

(1) 算法含义

- ◆ 层次聚类 (Hierarchical Clustering) 是一种经典的无监督学习算法，它通过构建一个嵌套的簇层次结构来组织数据。
- ◆ 与 K-means 不同，层次聚类不需要预先指定簇的数量，其目标是根据样本间的相似性（或距离）构建一个树状的聚类结构，允许从不同粒度观察聚类结果。
- ◆ 这种结构可以看作是一棵二叉树，叶子节点是单个样本，根节点包含所有样本，中间节点表示不同层次的簇。

2.5.2 层次聚类算法

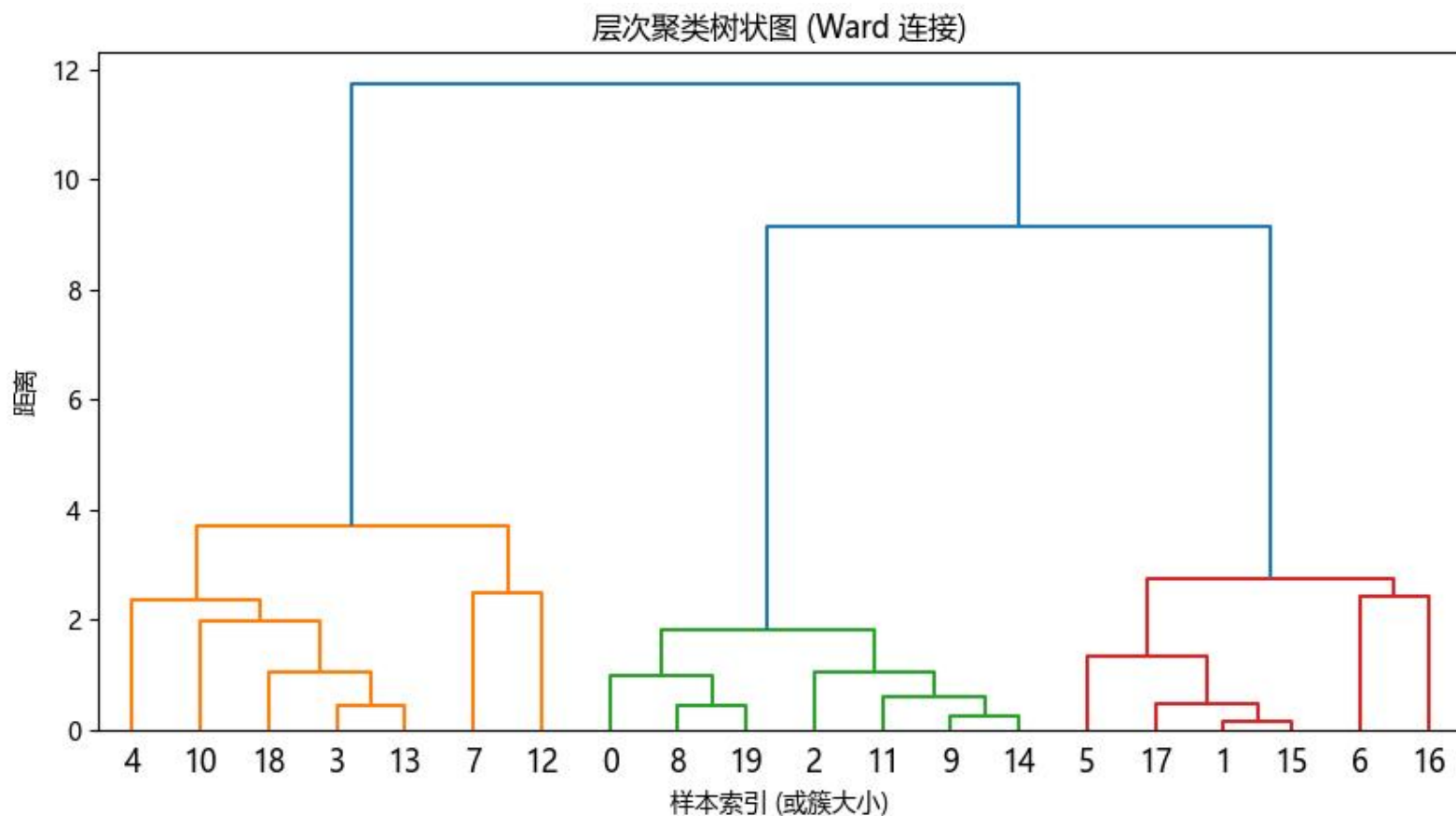
1. 相关概念

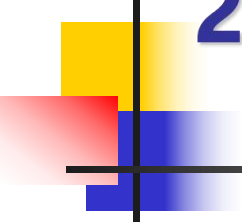
(2) 形象化比喻

- ◆ 以一个大家族族谱的绘制为例来理解层次聚类，其中：
 - ◆ 叶子：最底层的每一个个体，就是现在的你、你的兄弟姐妹、堂表亲等，这对应着层次聚类的每个数据点。
 - ◆ 小家庭：你和你的亲兄弟姐妹先聚在一起，形成一个小家庭。
 - ◆ 大家庭：你们这个小家庭再和叔叔一家合并，形成一个更大的家族。
 - ◆ 整个宗族：最后，所有家族不断合并，直到追溯到共同的祖先，形成一棵完整的族谱树。
- ◆ 从个体到整个宗族的合并过程，就是层次聚类中的凝聚法（自底向上）。从每个点开始，不断把“血缘”最近（距离最近）的样本或簇合并在一起。

2.5.2 层次聚类算法

1. 相关概念



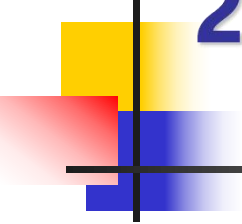


2.5.2 层次聚类算法

1. 相关概念

说明和分析:

- ◆ `inkage` 函数返回一个连接矩阵 Z ，每一行记录一次合并：前两列是合并的簇索引，第三列是合并距离，第四列是新簇包含的样本数。
- ◆ `dendrogram` 绘制树状图。
- ◆ `fcluster` 根据切割高度或指定簇数提取簇标签。



2.5.2 层次聚类算法

2. 基本类型

(1)凝聚（自底向上）层次聚类

- ◆ 初始时将每个样本视为一个独立的簇。
- ◆ 迭代合并最近的两个簇，直到所有样本合并为一个簇（或达到停止条件）。
- ◆ 最常用、最直观的方法。

(2)分裂（自顶向下）层次聚类

- ◆ 初始时将所有样本视为一个簇。
- ◆ 递归地将一个簇分裂成两个子簇，直到每个样本单独成簇（或达到停止条件）。
- ◆ 计算复杂度高，应用较少。

2.5.2 层次聚类算法

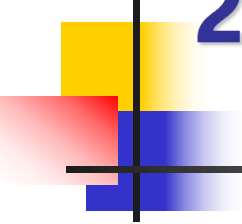
3. 算法核心要素

◆ (1) 距离度量。层次聚类需要定义两种距离：

- ◆ 样本间距离：衡量两个样本的相异度，常用的有欧氏距离、曼哈顿距离、余弦距离等。
- ◆ 类间距离（连接准则）：衡量两个簇之间的相异度，决定了合并哪些簇。

◆ (2) 连接准则。常用的连接准则（Linkage Criteria）：

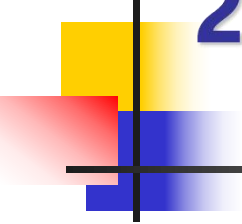
- ◆ 单链（Single Linkage）：两个簇之间的最近样本距离。
- ◆ 全链（Complete Linkage）：两个簇之间的最远样本距离。
- ◆ 平均链（Average Linkage）：两个簇之间所有样本对距离的平均值。



2.5.2 层次聚类算法

4. 算法主要步骤

- ◆ 输入：样本集 X ，距离度量 d ，连接准则 L 。
- ◆ 输出：树状图结构。
- ◆ (1)初始化：每个样本作为一个簇，计算所有簇间距离（对称矩阵）。
- ◆ (2)重复以下步骤直到只剩一个簇：
 - ◆ 找出距离最近的两个簇 C_i 和 C_j ；
 - ◆ 合并 C_i 和 C_j 为一个新簇 C_{ij} ；
 - ◆ 更新距离矩阵：删除与 C_i 、 C_j 相关的行和列，并计算新簇 C_{ij} 与其它簇的距离。
- ◆ (3)记录合并顺序及距离，构建树状图。



2.5.2 层次聚类算法

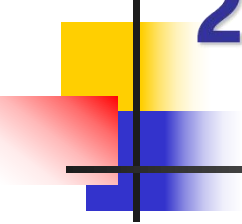
5. 优点和缺点

优点:

- ◆ 不需要预先指定簇数 K ，可通过树状图灵活选择。
- ◆ 能发现簇之间的层次关系，适合需要嵌套分类的场景。
- ◆ 对数据分布假设少，可处理任意形状的簇。

缺点:

- ◆ 时间复杂度高：（使用优先队列优化），不适合大规模数据。
- ◆ 对噪声和异常值敏感（特别是单链和全链）。
- ◆ 合并或分裂不可逆，一旦合并就不能撤销。

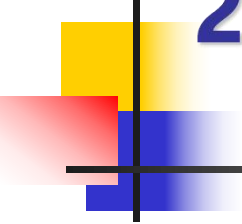


2.5.2 层次聚类算法

6. 应用场景

层次聚类算法常见应用场景如下：

- ◆ 基因表达数据分析：构建基因或样本的层次关系。
- ◆ 文档聚类：将文档组织成主题层次。
- ◆ 图像分割：根据颜色或纹理进行多尺度分割。
- ◆ 客户细分：从粗粒度到细粒度的客户分层。
- ◆ 社会网络分析：发现社区结构的嵌套关系。

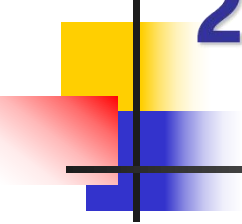


2.5.3 DBSCAN算法

1. 相关概念

(1) 算法含义

- ◆ DBSCAN (Density-Based Spatial Clustering of Applications with Noise) 是一种经典基于密度的聚类算法。
- ◆ DBSCAN 的核心思想：
 - ◆ 如果一个区域中点的密度超过某个阈值，就将其划分为一个簇。
 - ◆ 通过将紧密相连的样本（即密度足够高的区域）划分为同一簇，并将低密度区域中的样本视为噪声，从而完成聚类。
- ◆ DBSCAN 能够发现任意形状的簇，同时自动识别噪声点，并且不需要预先指定簇的数量，特别适合处理形状复杂、包含噪声的数据。



2.5.3 DBSCAN算法

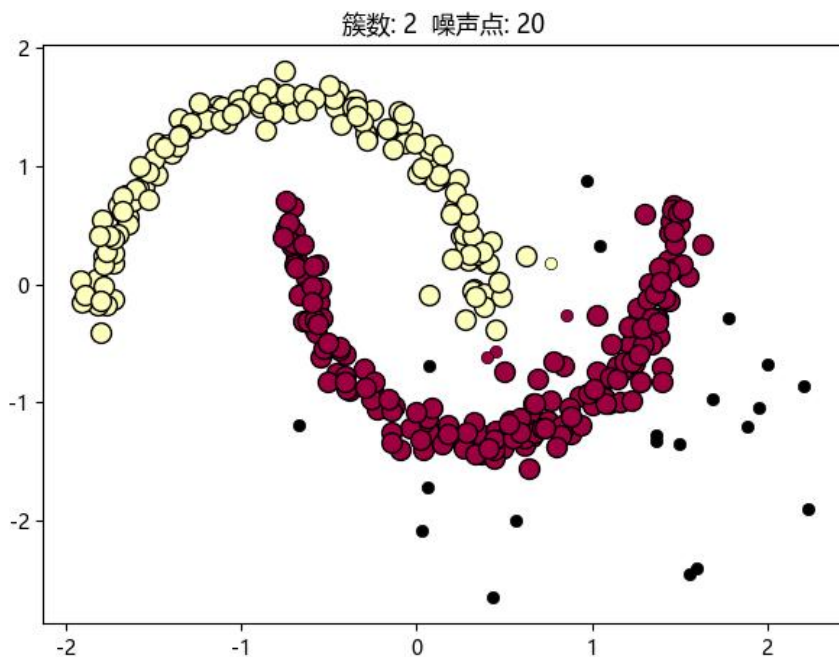
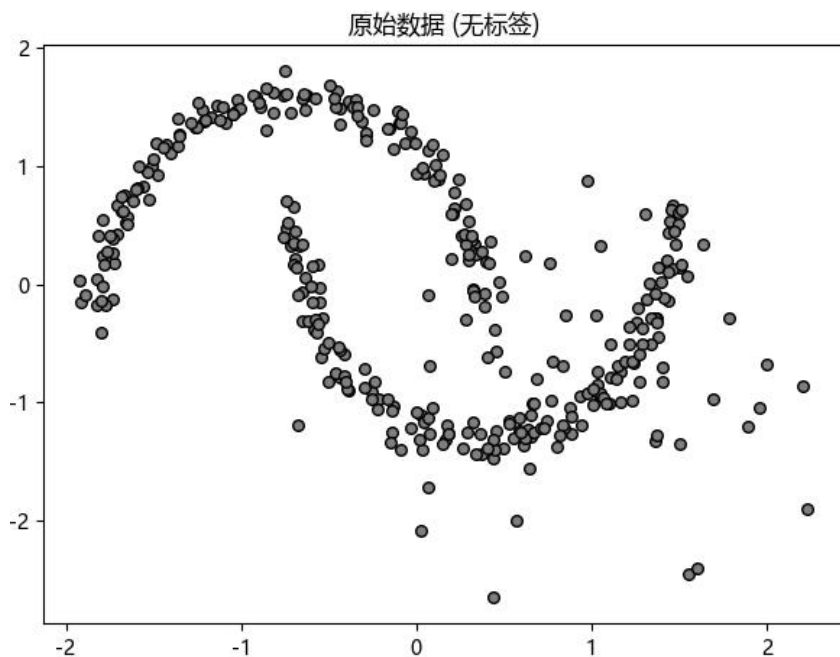
1. 相关概念

(2)形象化比喻：城市探秘 - 寻找热闹的商业区

- ◆ 核心点：有的商场和街道里面和周围人很多，很热闹。他们是这个热闹街区的“核心点”。
- ◆ 边界点：有的商场和街道里面和周围有些顾客，但不多。他们是“边界点”，也属于这个热闹街区的一部分。
- ◆ 噪声点：有的商场和街道在偏僻的小巷里，方圆百米见不到别人。他们是“噪声点”，不属于任何热闹街区。
- ◆ DBSCAN的工作就是：找到那些核心点，然后把这些商场和街道划为一个热闹街区。如果两个核心点的街区有重叠，就把它连成一片更大的商业区。

2.5.3 DBSCAN算法

1. 相关概念



2.5.3 DBSCAN算法

1. 相关概念

说明和分析：

- ◆ (1)左图：原始数据分布，看不出任何结构。
- ◆ (2)右图：
 - ◆ 不同颜色表示不同的簇（半月形被正确分开）。
 - ◆ 大圆点：核心点（密度高的区域）。
 - ◆ 小圆点：边界点（附着在簇边缘）。
 - ◆ 黑色点：噪声点（孤立点，不属于任何簇）。

2.5.3 DBSCAN算法

2. 算法主要步骤

DBSCAN 通过不断扩展密度可达的区域来发现簇，具体如下：

- ◆ (1)输入参数：邻域半径 ϵ 和最小样本数 MinPts。
- ◆ (2)标记核心点：遍历所有样本点，计算每个点的 ϵ -邻域内样本数，若 $\geq$ MinPts，则标记为核心点。
- ◆ (3)聚类：
 - ◆ 随机选择一个未访问过的核心点，开始一个新的簇。
 - ◆ 找出从该核心点出发的所有密度可达点（通过不断扩展核心点的邻域），构成一个簇。
 - ◆ 标记该簇中所有点（包括核心点和边界点）为已访问。
- ◆ (4)标记噪声：剩余未被归入任何簇的样本点即为噪声点。

2.5.3 DBSCAN算法

3. 算法核心参数

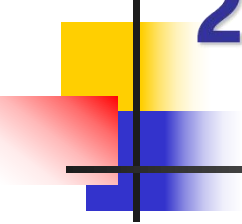
(1) 参数 ϵ (邻域半径)

- ◆ 定义：对于点 p ，其 ϵ 邻域是以 p 为圆心、 ϵ 为半径的圆形区域内包含的所有点。
- ◆ 作用： ϵ 直接决定了密度的尺度。
 - ◆ 如果 ϵ 太小，大部分点都无法形成核心点，导致大量点被标记为噪声，簇可能被过度分割。
 - ◆ 如果 ϵ 太大，不同簇会合并成一个，且噪声点也会被纳入簇中，丢失结构。
- ◆ 选择 ϵ 常用方法：k-距离图 (k-distance graph)。

2.5.3 DBSCAN算法

3. 算法核心参数

- ◆ (2)参数 **min_samples** (最小样本数)
- ◆ 定义：一个点被认定为核心点所需的最小邻居数（包括自身）。对于边界点或噪声点，无此要求。
- ◆ 作用：min_samples 控制了簇的最低密度，即一个核心点周围必须聚集多少个点才能被视为“密集区域”。
 - ◆ min_samples 越大，形成核心点的门槛越高，簇倾向于更密集、更少，噪声点可能增多。
 - ◆ min_samples 越小，越容易形成核心点，簇倾向于合并，但抗噪声能力下降。
- ◆ **min_samples选择**：一般设为数据维度的 2 倍。



2.5.3 DBSCAN算法

4. 优点和缺点

优点:

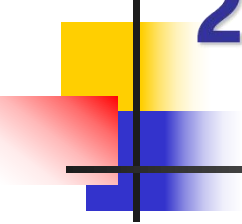
- ◆ 无需指定簇数：DBSCAN 自动根据密度确定簇的数量，无需像 K-Means 那样预先给定K。
- ◆ 能发现任意形状的簇：适合非球形、非凸的簇结构，如环形、S形等。
- ◆ 对噪声鲁棒：能自动识别并剔除噪声点，不受离群点干扰。
- ◆ 对数据顺序不敏感：结果稳定，不会因输入顺序不同而变化（除非存在边界点归属的模糊，但通常影响不大）。

2.5.3 DBSCAN算法

4. 优点和缺点

缺点:

- ◆ 对参数敏感: ϵ 和 MinPts 的选择直接影响结果, 调参较困难。
- ◆ 密度不均匀的数据效果差: 若数据集中存在密度差异较大的区域, 单一全局参数难以同时适应高密度和低密度簇。
- ◆ 高维数据失效: 随着维度增加, 距离度量趋于失效 (维度灾难), 密度定义变得模糊, 聚类质量下降。
- ◆ 计算复杂度较高: 若未使用空间索引, 时间复杂度为 $O(n^2)$; 若使用 R^* 树或 KD 树, 可降至 $O(n \log n)$, 但仍需注意内存消耗。

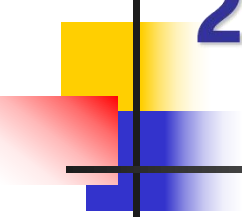


2.5.3 DBSCAN算法

5. 应用场景

DBSCAN 广泛应用于需要发现任意形状簇且存在噪声的场景：

- ◆ 地理空间数据分析：如基于 GPS 轨迹的停留点聚类、犯罪热点分析、地震震中分布等。
- ◆ 常检测：将噪声点视为异常，用于金融欺诈检测、网络入侵检测等。
- ◆ 图像分割：根据像素颜色或位置进行聚类，分离前景与背景。
- ◆ 生物信息学：基因表达数据分析、蛋白质序列聚类。
- ◆ 社交网络分析：基于用户互动密度的社区发现。

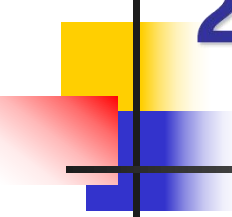


2.5.4 PCA算法

1. 相关概念

(1) 算法含义

- ◆ 主成分分析 (Principal Component Analysis, PCA) 是一种广泛应用于数据降维、特征提取和可视化的无监督学习方法。
- ◆ 核心思想：
 - ◆ 在高维数据中寻找一组新的正交基，这组基能够最大程度地解释数据的方差。
 - ◆ 通过线性组合原始特征，生成一组互不相关的新特征（主成分），其中第一个主成分捕获数据最大的方差，第二个主成分在正交于第一个主成分的方向上捕获剩余的最大方差，依此类推。
 - ◆ 最终可以选择前几个主成分来表示原始数据，达到降维的目的，同时尽可能保留原始数据的变异性。



2.5.4 PCA算法

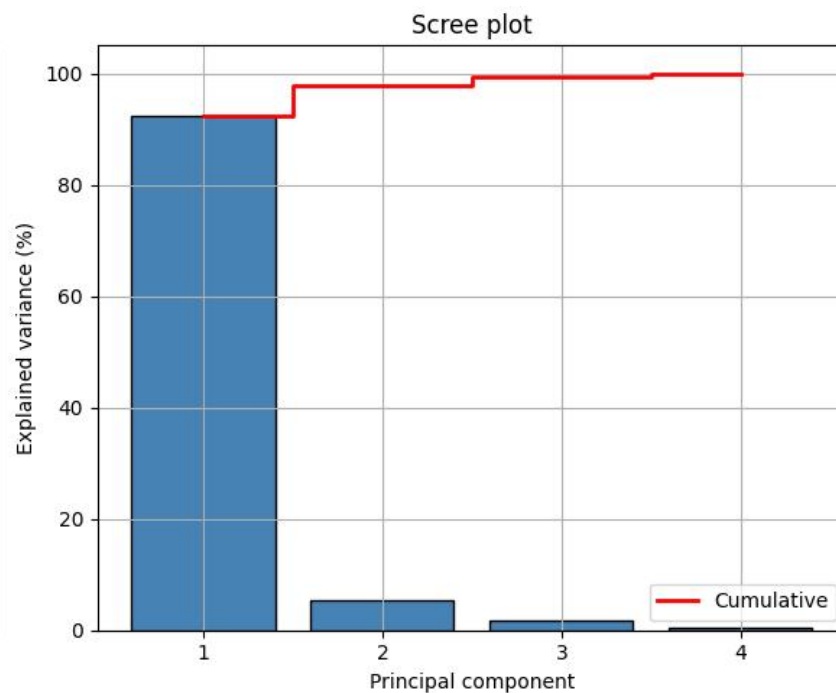
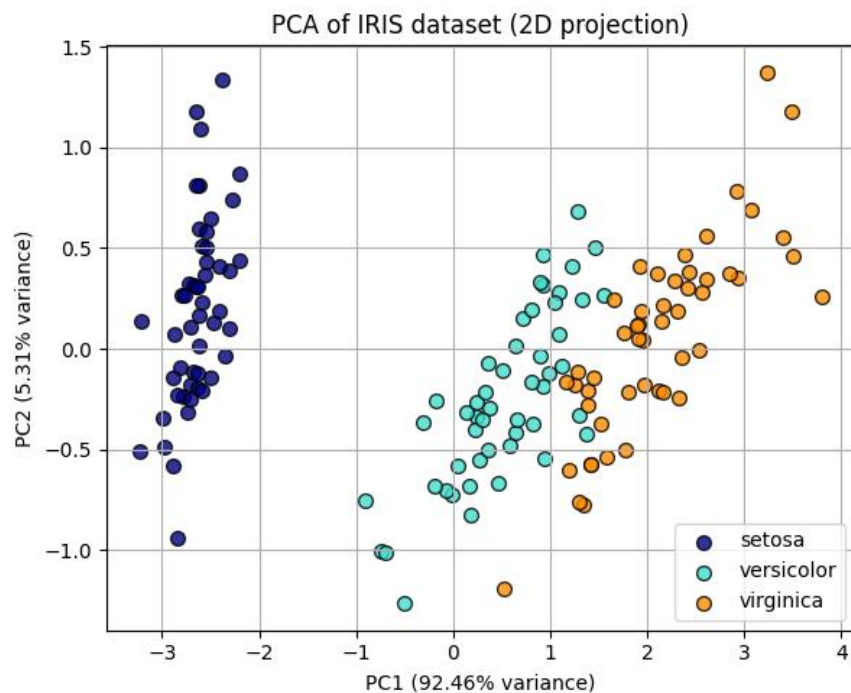
1. 相关概念

(2) 形象化比喻 - 看影子

- ◆ 一个人：是一个三维的物体（高维数据）。
- ◆ 地面上的影子：是二维的（降维后的数据）。
- ◆ 太阳的位置：决定了投影的方向（PCA要找的就是这个“最佳太阳角度”，让影子最能反映出这个人是谁）。
- ◆ 如果太阳在正头顶，影子只是一个圆点，信息几乎全丢了（这是糟糕的投影）。
- ◆ 如果太阳在侧面，影子清晰地显示出轮廓，别人一眼就能认出是谁（这是好的投影，保留了最大方差）。
- ◆ PCA就是“找影子”的艺术：找到一个最合适的角度，把高维物体的影子投射到低维平面上，让这个影子看起来最像原来的物体。

2.5.4 PCA算法

1. 相关概念



2.5.4 PCA算法

1. 相关概念

说明和分析:

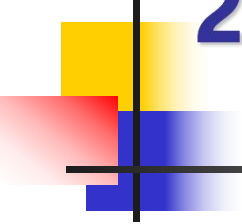
- ◆ 二维投影图。最常见的PCA可视化是将高维数据降维到2维，然后在散点图上观察样本的分布。如果数据有类别标签，还可以用不同颜色标记，看PCA是否能把不同类别的样本分开。图示解读：
 - ◆ 横轴：第一主成分（PC1），它解释了数据中最大的方差。
 - ◆ 纵轴：第二主成分（PC2），它解释了剩余方差中最大的部分，且与PC1正交。
 - ◆ 每个点代表一个样本，点的颜色代表原始类别（如果有的话）。
- ◆ 左侧散点图展示了鸢尾花数据在PC1和PC2平面上的投影。可以看到不同种类的花大致分布在不同的区域。

2.5.4 PCA算法

1. 相关概念

分析：

- ◆ (2)方差解释比例图（碎石图）。这个条形图或折线图展示了每个主成分所能解释的方差比例（或累积方差比例）。它帮助我们决定保留多少个主成分。图示解读：
 - ◆ 横轴：主成分序号（PC1, PC2, ...）。
 - ◆ 纵轴：每个主成分解释的方差比例（或累积比例）。
 - ◆ 通常我们会选择累积方差达到80%-90%的前k个主成分。
- ◆ 右侧条形图显示了每个主成分单独解释的方差比例，红色阶梯线是累积比例。例如，PC1+PC2可能已经解释了超过95%的方差，说明用2个主成分就能很好地代表原始4维数据。

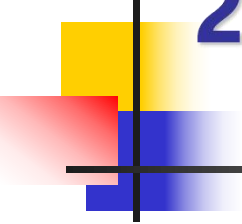


2.5.4 PCA算法

2. 算法主要步骤

标准PCA的实现通常包括以下几个步骤：

- ◆ 步骤1：数据标准化。需要将每个特征进行标准化（即减去均值并除以标准差），使每个特征的均值为0，方差为1。这样所有特征处于同一尺度。
- ◆ 步骤2：计算协方差矩阵（或直接使用SVD）。
- ◆ 步骤3：计算特征值和特征向量。
- ◆ 步骤4：选择主成分。按照特征值从大到小排序，前k个特征向量即为选定的主成分方向。
- ◆ 步骤5：将数据投影到主成分空间。每一列作为一个主成分得分，即原样本在新坐标系中的坐标。

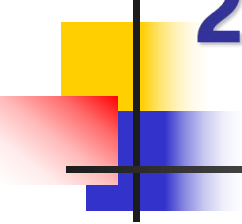


2.5.4 PCA算法

3. 优点和缺点

优点:

- ◆ 无参数限制: PCA的计算过程不涉及超参数 (除了降维维度), 完全由数据驱动。
- ◆ 降维同时保留主要信息: 通过最大化方差, 确保降维后的数据尽可能保留原始数据的变异性。
- ◆ 去相关: 主成分之间相互正交, 消除了原始特征间的线性相关性, 有利于后续分析。
- ◆ 可逆性: 如果保留所有主成分, 可以通过逆变换完全恢复原始数据 (不考虑标准化逆变换)。
- ◆ 计算简单: 基于线性代数, 有高效的数值算法 (如SVD)。



2.5.4 PCA算法

3. 优点和缺点

缺点:

- ◆ 线性假设: PCA只能捕捉数据的线性结构, 对于非线性流形数据效果不佳。
- ◆ 对异常值敏感: 由于基于方差, 极端值会显著影响主成分方向, 可能导致结果偏离。
- ◆ 主成分可解释性差: 主成分是原始特征的线性组合, 往往难以用实际意义解释。
- ◆ 方差小的成分不一定无用: 舍弃低方差成分可能丢失一些对分类等任务有用的信息。
- ◆ 标准化影响: 标准化与否对结果有较大影响, 需根据问题背景决定。

2.5.4 PCA算法

4. 应用场景

PCA算法常见应用场景如下：

- ◆ 数据降维：作为预处理步骤，减少特征数量，加快后续模型训练速度，降低过拟合风险。
- ◆ 可视化：将高维数据降至2D或3D，用散点图展示样本分布，观察聚类趋势或异常点。
- ◆ 特征提取：生成新的综合特征，用于分类、回归等任务。
- ◆ 噪声过滤：舍弃方差很小的主成分（通常对应噪声），保留主要成分，达到去噪效果。
- ◆ 异常检测：在低维空间中，重构误差较大的样本可能是异常点。
- ◆ 多重共线性处理：主成分之间彼此正交，消除了原始特征间的相关性，可用于回归建模（主成分回归）。